

Entregable 1 — Mapa de Arquitectura del Sistema

Proyecto final: Triage y Respuesta Automatizada de Consultas — ADC | Curso: IA Automation Avanzada, Coder House | Alumno: Sixto

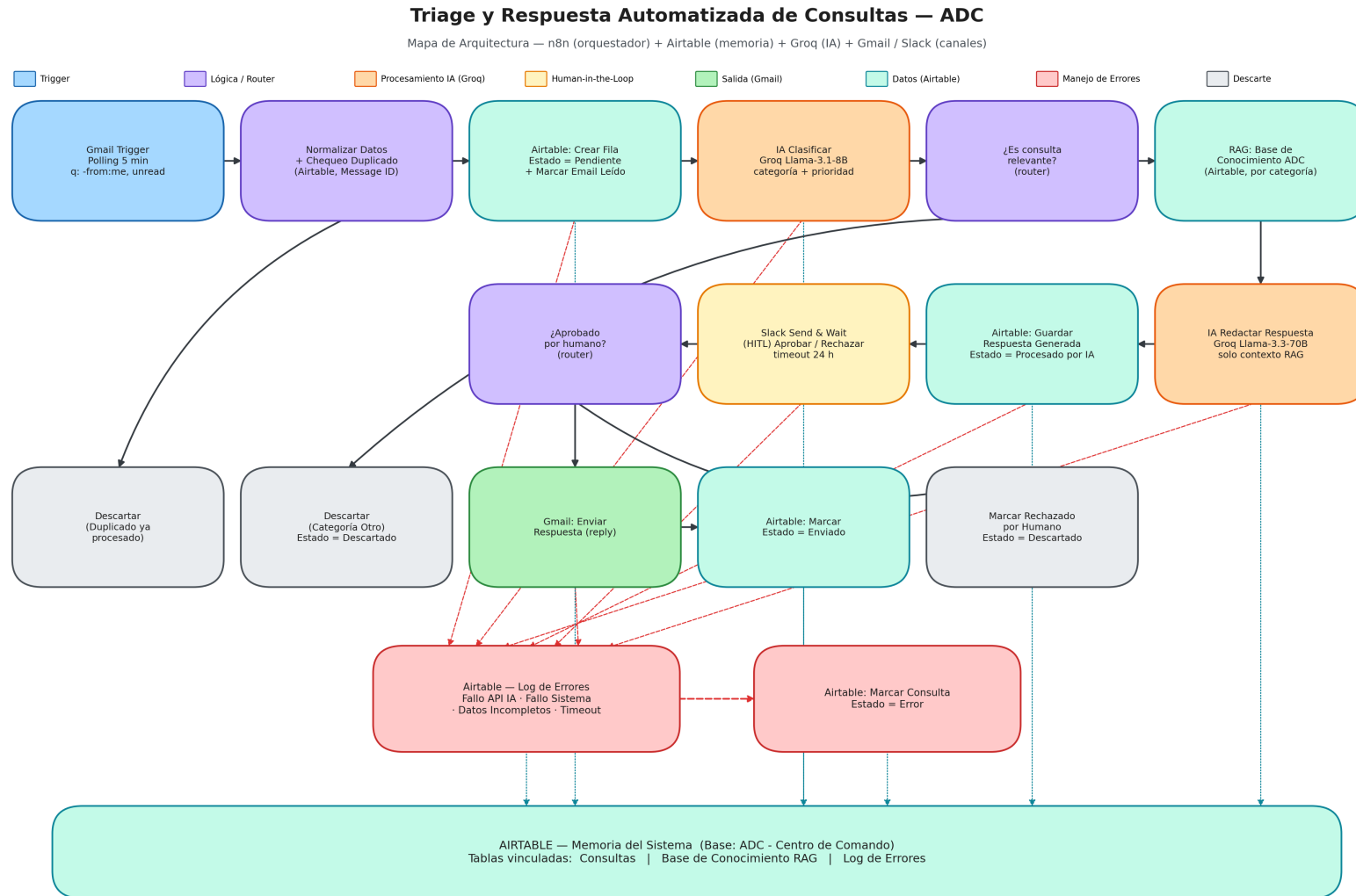


Diagrama generado a partir del workflow real de n8n y el esquema real de la base de Airtable del proyecto (ver Entregable 2).

Recorrido del flujo

El sistema arranca cuando llega un email nuevo a la casilla de ADC. n8n hace polling cada 5 minutos sobre Gmail, filtrando solo mensajes no leídos que no sean propios (-from:me) — esto evita que el bot se dispare a sí mismo o reprocese lo que ya contestó. El email se normaliza (remitente, asunto, mensaje, message_id) y se busca en Airtable si ese Message ID ya fue procesado antes; si es así, se descarta ahí mismo. Si es nuevo, se crea la fila en la tabla **Consultas** con Estado = *Pendiente* y se marca el email como leído de inmediato — esto es lo que corta el bucle: el próximo polling ya no lo va a volver a ver.

Con la fila creada, un agente de IA (Groq Llama-3.1-8B, el modelo más liviano del stack) clasifica la consulta en categoría (Comercial / Soporte Técnico / Otro) y prioridad, devolviendo un JSON validado por un parser estructurado. Si la categoría es “Otro”, el sistema corta acá y descarta — nunca llega a llamar al modelo más caro. Si es relevante, busca las directrices privadas en la Base de Conocimiento RAG de Airtable (filtradas por categoría) y se las pasa a un segundo agente de IA (Groq Llama-3.3-70B, más grande porque esta tarea sí requiere razonar sobre contexto) que redacta la respuesta **usando exclusivamente esas directrices**, nunca conocimiento externo.

La respuesta generada se guarda en Airtable (Estado = *Procesado por IA*) y recién ahí entra el punto de Human-in-the-Loop: se notifica en Slack con el borrador completo y el flujo queda pausado (nodo Send & Wait, timeout 24 h) hasta que una persona apruebe o rechace. Si aprueba, n8n recién ahí envía el email real por Gmail y marca la fila como *Enviado*. Si rechaza, la fila queda *Descartada* con la nota del motivo. En ningún punto el sistema le contesta a un cliente real sin ese visto bueno humano.

Todo paso que depende de una API externa (Airtable, Gmail o Slack) o de un modelo de IA tiene una salida de error dedicada: si falla, no corta la ejecución en seco, sino que registra el detalle en la tabla **Log de Errores** y marca la consulta como Estado = *Error*, para que quede trazable y se pueda reprocesar a mano.

Cobertura de los requisitos de arquitectura de la consigna

Requisito	Cómo se resuelve en este flujo
Trigger inteligente	Gmail Trigger con polling acotado (5 min) y filtro q: -from:me + solo no leídos. No procesa histórico al activarse.
Motor de IA con mapeo correcto	Salida de ambos agentes forzada a JSON válido (output parser estructurado); Max Tokens limitado a 200 (clasificar) y 500 (redactar).
Gestión de errores / resiliencia	10 nodos con onError → salida de error dedicada → Log de Errores + Estado = Error en Consultas. Cubre IA, Airtable, Gmail y Slack.
Human-in-the-loop	Slack Send & Wait antes de contactar al cliente final; requiere aprobación explícita (botón Aprobar / Rechazar).
Nodos nombrados, variables dinámicas, sin hardcodeo	Todos los nodos con nombre descriptivo en español; datos vía itemMatching(\$itemIndex); únicos literales son constantes de estado (ej. “Pendiente”).

Requisito	Cómo se resuelve en este flujo
Filtro anti-bucle infinito	Chequeo de duplicado por Message ID + email marcado como leído antes de procesar con IA.
Comparación de tipos correcta en filtros	Booleano vs booleano en la aprobación humana, string vs string en categoría — sin mezclas de tipo en los routers.